MongoDB Sharding Lab

By Shashank Kumar 24BAI10032

A. Before Starting

1. Stop Normal MongoDB Service

Open CMD as Administrator:

```
net stop MongoDB
```

Verify:

```
sc query MongoDB
```

Expected:

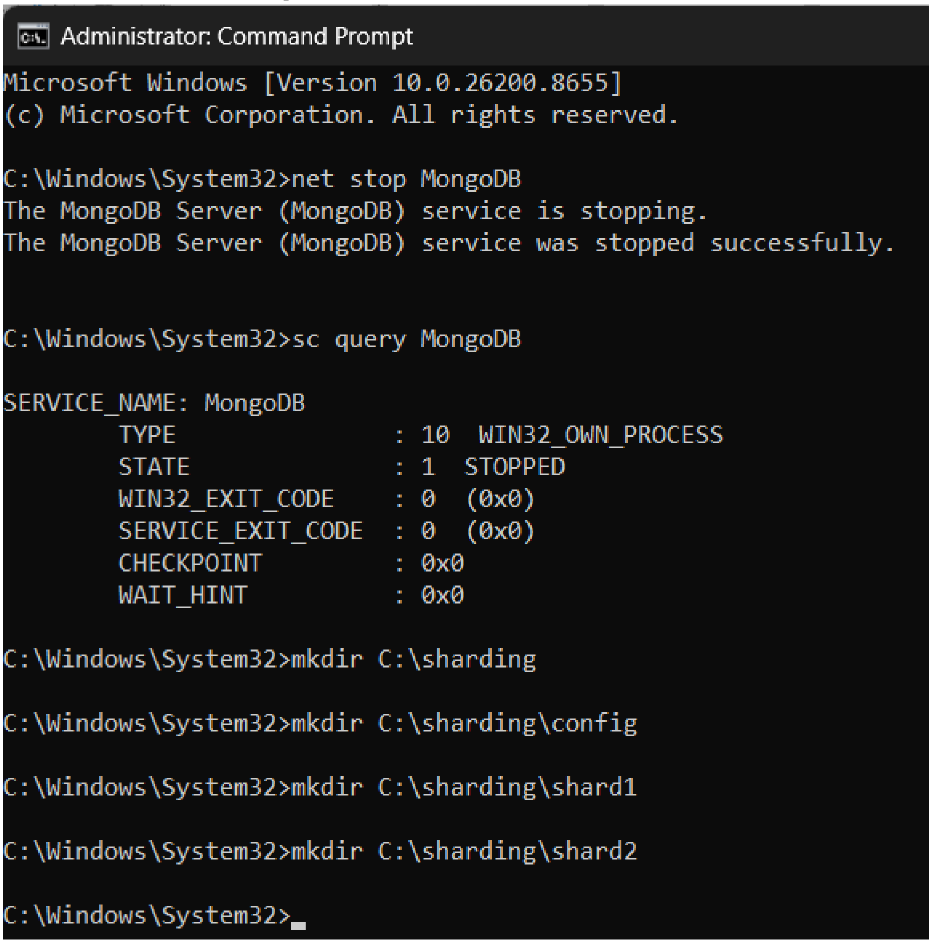
```
STATE : 1 STOPPED
```

B. Create Fresh Directories

(Optional: delete old sharding folders first)

```
mkdir C:\sharding
mkdir C:\sharding\config
```

```
mkdir C:\sharding\shard1
```




```
mkdir C:\sharding\shard2
```

C. Start Config Server

Open CMD Window #1

```
"C:\Program Files\MongoDB\Server\8.3\bin\mongod.exe" --configsvr --replSet
configReplSet --dbpath C:\sharding\config --port 27019
```

Leave window open.

D. Initialize Config Replica Set

Connect using Compass:

```
mongodb://127.0.0.1:27019/?directConnection=true
```

Open shell and run:

```
rs.initiate({
  _id: "configReplSet",
  configsvr: true,
  members: [
```

```
    { _id: 0, host: "localhost:27019" }
  ]
})
```

Verify:

```
rs.status()
```

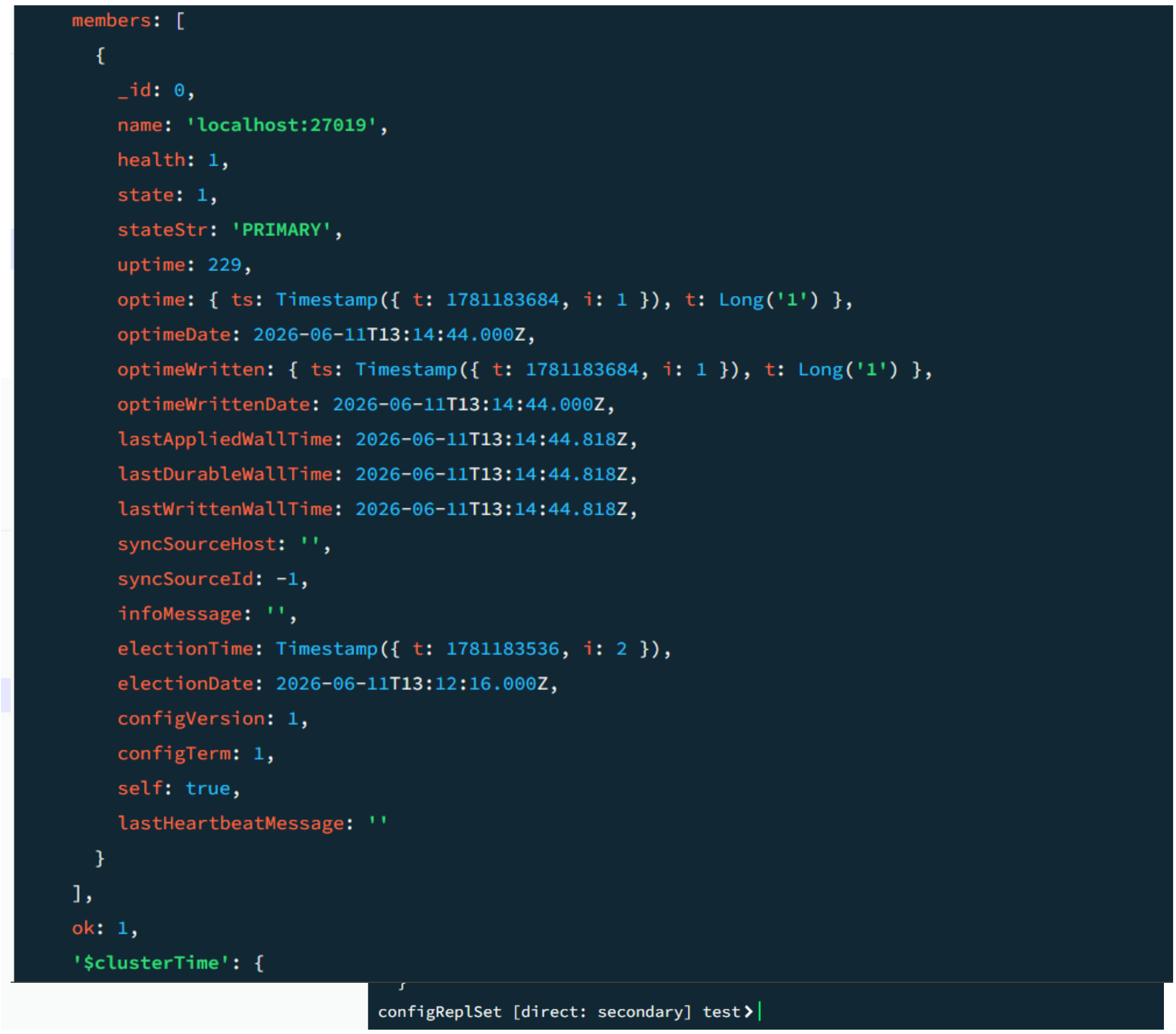
E. Start Shard 1

Open CMD Window #2

"C:\Program Files\MongoDB\Server\8.3\bin\mongod.exe" `--shardsvr --replSet shard1ReplSet --dbpath C:\sharding\shard1 --port 27011`

F. Initialize Shard 1

Connect:



```
  members: [
    {
      _id: 0,
      name: 'localhost:27019',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 229,
      optime: { ts: Timestamp({ t: 1781183684, i: 1 }), t: Long('1') },
      optimeDate: 2026-06-11T13:14:44.000Z,
      optimeWritten: { ts: Timestamp({ t: 1781183684, i: 1 }), t: Long('1') },
      optimeWrittenDate: 2026-06-11T13:14:44.000Z,
      lastAppliedWallTime: 2026-06-11T13:14:44.818Z,
      lastDurableWallTime: 2026-06-11T13:14:44.818Z,
      lastWrittenWallTime: 2026-06-11T13:14:44.818Z,
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1781183536, i: 2 }),
      electionDate: 2026-06-11T13:12:16.000Z,
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    }
  ],
  ok: 1,
  '$clusterTime': {
configReplSet [direct: secondary] test>
```

```
mongodb://127.0.0.1:27011/?directConnection=true
```

Run:

```
rs.initiate({
  _id: "shard1ReplSet",
  members: [
    { _id: 0, host: "localhost:27011" }
  ]
})
```

G. Start Shard 2

Open CMD Window #3

```
"C:\Program Files\MongoDB\Server\8.3\bin\mongod.exe" --shardsvr --replSet shard2ReplSet --dbpath C:\sharding\shard2 --port 27012
```

H. Initialize Shard 2

Connect:

```
mongodb://127.0.0.1:27012/?directConnection=true
```

Run:

```
rs.initiate({
  _id: "shard2ReplSet",
  members: [
    { _id: 0, host: "localhost:27012" }
  ]
})
```

I. Start mongos Router

Open CMD Window #4

```
"C:\Program Files\MongoDB\Server\8.3\bin\mongod.exe" --configdb configReplSet/localhost:27019 --port 27018
```

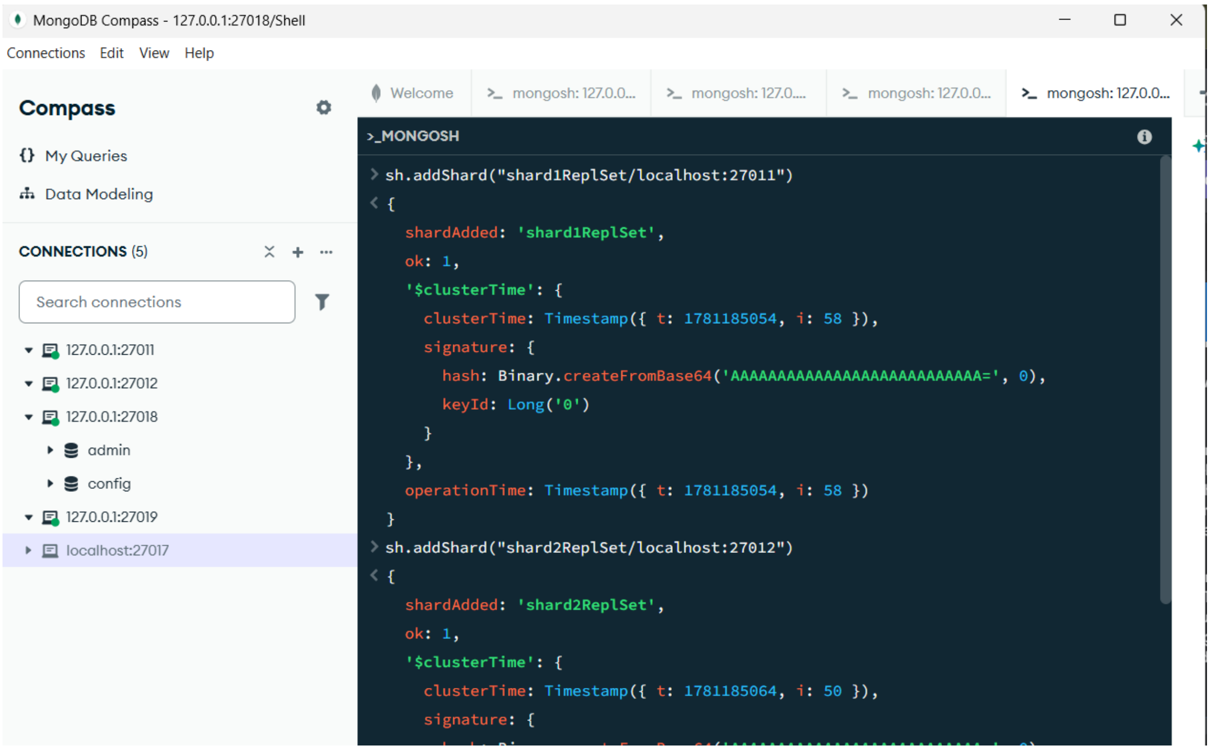
Leave running.

J. Connect to mongos

Compass:

```
mongodb://127.0.0.1:27018
```

K. Add Shards



Run:

```
sh.addShard("shard1ReplSet/localhost:27011")
sh.addShard("shard2ReplSet/localhost:27012")
```

Verify:

```
sh.status()
```

L. Enable Sharding

```
sh.enableSharding("eCommerceDB")
```

```
> sh.enableSharding("eCommerceDB")
< {
    ok: 1,
    '$clusterTime': {
      clusterTime: Timestamp({ t: 1781185144, i: 21 }),
      signature: {
        hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
        keyId: Long('0')
      }
    },
    operationTime: Timestamp({ t: 1781185144, i: 21 })
  }
[mongos] test>
```

M. Shard Collection

```
sh.shardCollection(
  "eCommerceDB.orders",
  { customer_id: "hashed" }
```

```
> sh.shardCollection(
    "eCommerceDB.orders",
    { customer_id: "hashed" }
  )
< {
    collectionsharded: 'eCommerceDB.orders',
    ok: 1,
    '$clusterTime': {
      clusterTime: Timestamp({ t: 1781185172, i: 24 }),
      signature: {
        hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
        keyId: Long('0')
      }
    },
    operationTime: Timestamp({ t: 1781185172, i: 23 })
  }
[mongos] test>
```

```
)
```

N. Insert Test Data

```
use eCommerceDB
for(let i=0;i<1000;i++){
  db.orders.insertOne({
    customer_id:"cust_"+i,
    amount:Math.random()*100
  })
}
```

O. Verify Distribution

Check:

```
sh.status()
```

or

```
db.orders.getShardDistribution()
```

```
>_MONGOSH
> db.orders.getShardDistribution()
<
Shard shard1ReplSet at shard1ReplSet/localhost:27011

{
  data: '0B',
  docs: 0,
  chunks: 1,
  'estimated data per chunk': '0B',
  'estimated docs per chunk': 0
}
---
Shard shard2ReplSet at shard2ReplSet/localhost:27012

{
  data: '0B',
  docs: 0,
  chunks: 1,
  'estimated data per chunk': '0B',
  'estimated docs per chunk': 0
}
```

Expected:

```
Shard1 ≈ 50%
Shard2 ≈ 50%
```

Shutdown Procedure

1. Stop mongos

In Window #4:

```
Ctrl + C
```

---

2. Stop Shard 1

In Window #2:

```
Ctrl + C
```

---

3. Stop Shard 2

In Window #3:

```
Ctrl + C
```

---

4. Stop Config Server

In Window #1:

```
Ctrl + C
```

---

5. Verify All Stopped

```
netstat -ano | findstr 27018
netstat -ano | findstr 27019
netstat -ano | findstr 27011
netstat -ano | findstr 27012
```

No LISTENING entries should remain.

Return to Normal MongoDB

Start the Windows service again:

```
net start MongoDB
```

Verify:

```
sc query MongoDB
```

Expected:

```
STATE : 4 RUNNING
```

Check:

```
netstat -ano | findstr 27018
```

Expected:

```
LISTENING
```

Compass can now connect normally to:

```
mongodb://localhost:27018
```

```
Administrator: Command Prompt
  TCP    127.0.0.1:62250        127.0.0.1:27019        TIME_WAIT       0
  TCP    127.0.0.1:62453        127.0.0.1:27019        TIME_WAIT       0
  TCP    127.0.0.1:62551        127.0.0.1:27019        TIME_WAIT       0

C:\Windows\System32>netstat -ano | findstr 27011
  TCP    127.0.0.1:51774        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:58319        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:58325        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:62209        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:62218        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:62219        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:62221        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:62241        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:62243        127.0.0.1:27011        TIME_WAIT       0
  TCP    127.0.0.1:62252        127.0.0.1:27011        TIME_WAIT       0

C:\Windows\System32>netstat -ano | findstr 27012
  TCP    127.0.0.1:51775        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:60072        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:60078        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62225        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62236        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62237        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62239        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62244        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62245        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62246        127.0.0.1:27012        TIME_WAIT       0
  TCP    127.0.0.1:62253        127.0.0.1:27012        TIME_WAIT       0

C:\Windows\System32>net start MongoDB
The MongoDB Server (MongoDB) service is starting..
The MongoDB Server (MongoDB) service was started successfully.


C:\Windows\System32>netstat -ano | findstr 27017
  TCP    127.0.0.1:27017        0.0.0.0:0              LISTENING       20600

C:\Windows\System32>
```